pandas Cheat Sheet

pandas 3.1 · Python ≥ 3.12 · checked October 2026 · Irurueta Docs · original summary

1. Input / output

Format	Read	Write
CSV	`read_csv`	`to_csv(index=False)`
Parquet	`read_parquet`	`to_parquet`
Feather, ORC	`read_feather`, `read_orc`	`to_feather`, `to_orc`
Excel	`read_excel` (openpyxl, calamine)	`to_excel`
JSON, HTML, XML	`read_json`, `read_html`, `read_xml`	`to_json`, `to_html`, `to_xml`
SQL	`read_sql(q, engine)`	`to_sql(if_exists=)`
Iceberg	`read_iceberg`	`to_iceberg`

- `read_csv`: `usecols`, `dtype`, `parse_dates`, `date_format`, `na_values`, `nrows`, `chunksize`, `sep=r"\s+"`, `dtype_backend="pyarrow"` or `"numpy_nullable"`.
- 3.1: C engine reads local files ≥ 5 MB in parallel (`mode.max_threads`; 1 = serial).
- Prefer Parquet (keeps dtypes, fast). `engine="fastparquet"` deprecated. Pickle only for trusted data.
- `read_excel`: `sheet_name`, `header`, `usecols`; `xlrd/pyxlsb` deprecated.

2. Selection: loc, iloc, pd.col

Goal	Code
Column / columns	`df["a"]`, `df[["a","b"]]`
By label (inclusive)	`df.loc["r1":"r3", "a":"c"]`
By position (end excl.)	`df.iloc[0:3, [0, 2]]`
Boolean mask	`df.loc[df.x > 1, "a"]`, `df.iloc[mask]`
Expression (3.0+)	`df.loc[pd.col("x") > 1]`
Select + compute (3.1)	`df.select("a", t=pd.col("x")*2)`
Filter rows (3.1)	`df.filter(cond=pd.col("x")>1)`
Add columns	`df.assign(z=pd.col("x")+1)`
Query string	`df.query("x > 1 and k == 'a'")`
Scalar fast	`df.at["r","a"]`, `df.iat[0,1]`
By dtype	`select_dtypes(include="number")`

- `pd.col` is deferred: it works wherever a `lambda df: ...` was accepted. `select` and `filter(cond=)` do not exist on 3.0.
- `.ix`, `get_value` are gone. Always `loc` for set: `df.loc[mask, "a"] = 0`.
- Numeric keys in `ser[-1]` never fall back to position: use `iloc[-1]`.

3. Cleaning and missing data

Task	Code
Detect	`isna()`, `notna()`, `isna().sum()`
Drop	`dropna(subset=, how=, thresh=)`
Fill	`fillna({"x": 0})`, `ffill()`, `bfill()`
Interpolate	`interpolate(method="time")`
Replace	`replace({"?": pd.NA})`, `where`, `mask`, `clip`
Duplicates	`duplicated(subset)`, `drop_duplicates(subset, keep=)`
Types	`astype("Int64")`, `pd.to_numeric(errors="coerce")`, `convert_dtypes()`
Text	`.str.strip()`, `.str.contains(pat)`, `.str.replace(p, r, regex=True)`
Bins, categories	`cut`, `qcut`, `astype("category")`

- Missing markers: `NaN` (float, `str`), `pd.NA` (nullable), `NaT` (dates). Test with `pd.isna`, never `== np.nan`.
- Strings are the `str` dtype by default (`object` only if forced). `astype(str)` keeps missing values missing.
- Incompatible assignment now raises instead of upcasting (PDEP-6).

4. Merge, concat and reshape

Need	Code
SQL-style join	`df.merge(o, on="k", how="left", validate="m:1", indicator=True)`
Different keys	`left_on=, right_on=, suffixes=("_l","_r")`
Join on index	`df.join(o, how="inner")`
Anti-join (3.0)	`how="left_anti"` or `"right_anti"`
Nearest key	`pd.merge_asof(a, b, on="t", by="id")`
Stack frames	`pd.concat(list, ignore_index=True)`
Side by side	`pd.concat([a, b], axis=1, join="inner")`
Long from wide	`melt(id_vars=, value_vars=)`
Wide from long	`pivot(index=, columns=, values=)`, `pivot_table(aggfunc=)`
Index to columns	`stack()`, `unstack()`, `explode("col")`

- `how`: inner, left, right, outer, cross, left_anti, right_anti. `validate` catches key explosions early.
- `append` is gone: collect pieces in a list, concat once.
- `pivot` fails on duplicate keys; `pivot_table` aggregates them.

5. GroupBy

`df.groupby(["k1","k2"], as_index=False, dropna=False, sort=False)`

Pattern	Result
`.agg(n=("x","size"), m=("x","mean"))`	named aggregation, one row per group
`["x"].transform("mean")`	same length as input (group means, z-scores)
`.filter(lambda g: len(g) > 2)`	whole groups kept or dropped
`.apply(f, include_groups=False)`	flexible, slowest
`.head(2)`, `.nth(0)`, `.cumcount()`	position inside the group
`.size()` vs `.count()`	rows vs non-missing values
`pd.Grouper(key="d", freq="QE")`	time bins as a key

- Reductions accept `skipna` (3.0). Categorical keys default to `observed=True`.
- `groupby(axis=1)` removed: transpose, group, transpose.
- Prefer built-in reductions (`sum`, `mean`, `nunique`) over lambdas: they are vectorised.

6. Windows

Kind	Code
Rolling (fixed)	`s.rolling(7, min_periods=3).mean()`
Rolling (time)	`s.rolling("30D").sum()` (sorted DatetimeIndex)
Expanding	`s.expanding().max()`
Exponential	`s.ewm(span=10).mean()`, `halflife=`, `alpha=`
Custom	`rolling(5).apply(f, raw=True)`
Shift, diff	`shift(1)`, `diff()`, `pct_change(fill_method=None)`

- Also `first`, `last`, `nunique` on rolling (3.0). `center=True`, `closed=` control the label.

Inspect and tidy quickly

- `info()`, `describe(include="all")`, `nunique()`, `memory_usage(deep=True)`, `head()`, `sample(n, random_state=0)`.
- `value_counts(normalize=True)`, `nlargest(3, "x")`, `corr(numeric_only=True)`, `rank()`.
- `sort_values(["k","x"], ascending=[True, False])`, `rename(columns=)`, `set_index("k")`, `reset_index(drop=True)`.
- `pd.option_context("display.max_columns", None)` changes display only inside the block.

7. Time series and aliases

Code	Meaning
`pd.to_datetime(s, format="%Y-%m-%d")`	parse; `errors="coerce"`
`pd.date_range("2024", periods=4, freq="ME")`	generate
`s.resample("QE").sum()`, `asfreq("D")`	re-bin, re-index
`s.tz_localize("UTC").tz_convert("Europe/Madrid")`	zoneinfo zones
`s.dt.year`, `.dt.day_of_week`, `.dt.floor("h")`	components
`s.dt.as_unit("ns")`	change resolution

Alias	Unit	Alias	Unit
`D B`	day, business	`h min s`	hour, minute, second
`W W-MON`	week	`ms us ns`	sub-second
`ME MS`	month end, start	`BME SME`	business, semi month end
`QE QS`	quarter	`YE YS`	year

- Resolution is inferred (strings give `us`), no longer always `ns`; integer casts differ by 1000.
- `M Q Y A H T S L U N` raise `ValueError`; Period frequencies still use `M Q Y`.
- `Day` is a calendar day across DST, not 24 hours.

8. Copy-on-Write rules



- Every result of an indexing or method call behaves as a copy; no `SettingWithCopyWarning`.
- Chained assignment is a no-op and warns (`ChainedAssignmentError`): `df["a"][0] = 1` and `df[mask]["a"] = 1` do not update `df`. Write `df.loc[mask, "a"] = 1`.
- `inplace=True` on a selection (`df["a"].fillna(0, inplace=True)`) never reaches the parent. Assign: `df["a"] = df["a"].fillna(0)`.
- `to_numpy()` / `.values` can be read-only; `.copy()` for a writable array. `Index.values` read-only (3.1).
- Defensive `.copy()` calls are unnecessary. `mode.copy_on_write` option is deprecated (always on).

9. Migration to 3.x (3.1)

Old	Now
`df.append(x)`	`pd.concat([df, x])`
`applymap(f)`	`df.map(f)`
`fillna(method="ffill")`	`ffill()`
`pd.value_counts(x)`	`x.value_counts()`
`dtype == object` for text	`is_string_dtype(s)`, `== "str"`
`to_datetime(errors="ignore")`	catch, or `"coerce"`
`sort_values(inplace=True)`	`df = df.sort_values()`
`str.replace(".", "-")`	literal now; `regex=True`

- 3.1: `inplace=` warns on drop, rename, reset_index, set_index, dropna, drop_duplicates, sort_values, sort_index, query, eval (`Pandas4Warning`).
- Python ≥ 3.11, NumPy ≥ 2.0.2, PyArrow ≥ 16 (optional). Releases: 3.0 2026-01-21, 3.1 2026-10-22; 3.2 is upcoming.
- Upgrade path: 2.3, fix warnings, then 3.0, 3.1; pin `pandas>=3.1,<4`.

Changed since older tutorials: no `SettingWithCopyWarning`, chained assignment does nothing · text is `str`, not `object` · `ME QE YE h min` replace `M Q Y H T` · `append`, `applymap`, `.ix`, `fillna(method=)` removed · datetimes are no longer always nanoseconds · `inplace=True` is deprecated and never saved memory · PyArrow stays optional · `pd.col`, `select`, anti-joins are new.

Original summary, verified by running examples on pandas 3.1.0 / Python 3.13. Full guide: albertoirurueta.github.io/docs → Data Science → Pandas.